

Anexo — Mejoras y Corrección de Bugs

Escuela Politécnica Nacional · Ingeniería de Software GR1 · Juegos Interactivos

Docente: Ing. Vicente Adrián Egüez Sarzosa · 24 de julio de 2026

Equipo: Dylan Granizo Ayala · Anderson Cango · Dayana Lema

Juego en vivo: <https://dylan1410.github.io/NeonEscape/>

Documento de respaldo del examen práctico: detalla, para cada mejora y bug, el **estado anterior** y la **solución aplicada**.

PARTE A · LAS 9 MEJORAS DEL EXAMEN (3 POR INTEGRANTE)

1. Coyote time + jump buffer DYLAN · N1

ANTES

El salto se sentía rígido: al caer del borde de una plataforma no respondía, y el input de salto justo antes de aterrizar se perdía.

DESPUÉS

`coyoteTimer` (100 ms de gracia tras dejar el suelo) y `jumpBufferTimer` (120 ms de anticipación) en `update()`, conservando el doble salto.

2. SFX y feedback de recolección DYLAN · N1

ANTES

Recoger núcleos y saltar no producían ningún sonido ni impacto perceptible.

DESPUÉS

`collect.wav` al recolectar y un tick sonoro al saltar, con micro-animación de escala en el núcleo.

3. Timer jugable + aviso de peligro DYLAN · N1

ANTES

Temporizador de 25 s (muy ajustado) y sin ninguna señal de que el tiempo se agotaba.

DESPUÉS

Tiempo a 40 s; el contador **parpadea en rojo** cuando quedan ≤ 10 s (tween de color/alpha).

4. Fix de progresión entre niveles

ANDERSON · N2

ANTES

Completar el Nivel 2 llamaba `unlockLevel(2)` y la tecla **N** reiniciaba el propio nivel en vez de avanzar.

DESPUÉS

`unlockLevel(3)` y `scene.start('Scene_Nivel3')` en la rama de la tecla siguiente.

5. Assets fantasma + SFX de dash

ANDERSON · N2

ANTES

Errores **404** en consola por `joltDash_strip.png` y `speed_boost.wav` inexistentes; el dash no tenía sonido.

DESPUÉS

Cargas y código muerto eliminados (sin 404); SFX de dash con un audio ya existente.

6. Feedback de daño en pinchos

ANDERSON · N2

ANTES

Morir en los pinchos no tenía ningún impacto audiovisual.

DESPUÉS

`hit.wav` + **flash rojo** de cámara al pisar pinchos, antes de la pantalla de derrota.

7. Música de nivel con fade

DAYANA · N3

ANTES

El Nivel 3 no tenía música, inconsistente con los Niveles 1 y 2.

DESPUÉS

`bgm.wav` en loop con *fade-in/out* y detención limpia al salir de la escena.

8. SFX de combate

DAYANA · N3

ANTES

Disparar, destruir drones y ganar/perder eran acciones mudas.

DESPUÉS

SFX en `shoot()` y `hitEnemy()`, más jingles `win.wav` / `lose.wav`.

9. Menú de pausa + limpieza de código muerto

DAYANA · N3

ANTES

El Nivel 3 no tenía menú de pausa; existían escenas huérfanas `Scene_Game.js` y `Scene_End.js`.

DESPUÉS

Pausa con `ESC` (overlay + `physics.pause()`);
archivos huérfanos eliminados.

PARTE B · CORRECCIÓN DE BUGS ESTRUCTURALES (PLAYTESTING)

Fallos descubiertos durante las corridas de playtesting que dejaban niveles incompletables. Corregidos como parte del pulido.

B1. Navegación: el Nivel 2 mostraba el mapa del Nivel 1 N2

ANTES

El Nivel 2 cargaba su tilemap con la clave de caché `mapa_nivel1` (igual que el Nivel 1); Phaser servía el mapa ya cacheado del Nivel 1.

DESPUÉS

Clave renombrada a `mapa_nivel2` en la carga y en `make.tilemap`. Navegación Menú→N1→N2→N3 verificada.

B2. Mapa del Nivel 2 injugable N2

ANTES

El spawn quedaba **dentro de bloques sólidos** y el piso era un **muro continuo de pinchos** sin ruta segura: imposible avanzar.

DESPUÉS

Mapa rediseñado: piso continuo + **3 tiras de pinchos** de 2 tiles cruzables con dash/salto; pickups, núcleos y portal reubicados; timer a 45 s; plataforma móvil retirada.

B3. Pickups de energía incolectables N2

ANTES

El cuerpo de física del pickup estaba dentro de un `container` en posición local (0,0); Arcade Physics ignora la transformación del container, así que el cuerpo quedaba en (0,0) y el overlap nunca ocurría.

DESPUÉS

Glow/icono/cuerpo colocados directamente en coordenadas del mundo, con hitbox generoso. Se recogen correctamente.

B4. Dash "infinito" N2

ANTES

Cooldown de 0.8 s con auto-recarga: se podía spamear y los pickups no servían de nada.

DESPUÉS

Cooldown de **3.5 s**; los pickups de energía recargan el dash al instante, dándoles propósito.

B5. Portal del Nivel 2 inalcanzable N2

ANTES

`portalY = 360` : la caja de meta quedaba por debajo de la superficie donde el jugador se para ($y \approx 320$), sin overlap posible.

DESPUÉS

`portalY = 296` , alineado con la superficie; se toca caminando.

B6. Las balas no se disparaban N3

ANTES

`setVelocityX()` se ejecutaba **antes** de agregar la bala al physics group, que reseteaba la velocidad: la bala nacía con velocidad 0 y quedaba pegada al jugador. Sin balas no se mataban drones → portal bloqueado.

DESPUÉS

Se agrega la bala al grupo primero y la velocidad se fija al final (igual para las torretas).

B7. Piso tóxico letal (sludge) N3

ANTES

Una franja verde en el piso mataba al tocarla sin una razón de diseño clara, bloqueando el paso.

DESPUÉS

Sludge eliminado por completo (cargas, creación, colisión y métodos). Piso transitable de punta a punta.

B8. Drones inalcanzables N3

ANTES

Los saltos entre plataformas eran de ~ 125 px y el salto de Kael llegaba a ~ 126 px (al límite): las plataformas altas eran inalcanzables y con ellas 2 drones.

DESPUÉS

Salto de Kael `-450 → -520` (alcanza ~ 169 px), con margen para escalar toda la torre y destruir los 5 drones.

B9. Contacto con enemigos sin daño N3

ANTES

Los hitboxes de drones y torretas eran pequeños, así que el contacto con el jugador casi nunca registraba daño.

DESPUÉS

Hitboxes ampliados (drones `26×28`, torretas `28×30`): los disparos aciertan más fácil y el contacto hace daño de forma fiable.

B10. Portal del Nivel 3 inalcanzable N3

ANTES

Portal en `y=505`, ~45 px sobre el piso; la caja de meta quedaba encima del jugador parado y no había overlap.

DESPUÉS

Bajado a `y=525` con cuerpo más alto y `refreshBody()`; se completa caminando al portal.